



**UNIVERSIDAD COMPLUTENSE
MADRID**

Actividad de Minería de datos y modelización predictiva - 2

Autor: Gerardo Fructuoso Vidal-Aragón

Profesor/a: Juana María Alonso Revenga

Índice

1. Introducción	2
2. Introducción a los datos usados.....	3
3. Gráficos y comportamiento estacional	5
4. Dividir datos en train y test	8
5. Modelos de suavizado	9
6. Correlogramas y modelo Arima manual	13
7. Modelo ARIMA automático.....	18
8. Expresión algebraica	19
9. Predicciones del modelo ARIMA	20
10. Comparación ARIMA y suavizado.....	22

1. Introducción

En esta asignatura de Minería de datos y modelización predictiva (parte II), hemos llegado a ver para poder realizar modelos de datos Simples y suavizados hasta usar el modelo ARIMA.

Hemos dividido el conjunto de datos en series (que es con lo que se trabaja para crear estos modelos).

Una vez hecho esto, hemos creado los siguientes modelos:

- Suavizados: Hemos aprendido a utilizar el modelo de suavizado de Holt-Winters, modelo de alisado simple y el modelo de alisado doble de Holt
- Modelo ARIMA: Aquí hemos aprendido previamente a diferenciar los modelos autoregresivos y los de medias móviles para por último utilizar el modelo ARIMA (tanto hacerlo de forma automática como de forma manual).

Una vez realizada una breve introducción al temario, se procede a realizar la tarea.

2. Introducción a los datos usados

Los datos que han sido utilizados para realizar esta actividad, son descargados de la página del ayuntamiento de Madrid. Estos datos han sido descargados según se indica en la guía que viene adjuntada en la documentación.

El filtro utilizado para descargar los datos es el siguiente:

FILTROS DE BÚSQUEDA

Nombre de la serie	Número de serie
temperatura	
Nombre del epígrafe	Código del epígrafe

Y se ha cogido la última serie ya que ponía para comparar en los últimos 30 años:

2.2. Temperatura media, Precipitación y su comparación con la de los últimos 30 años. (Observatorio de Retiro)

Para descargar esta serie (al tener como máximo el año 2024) para coger unos 150 datos, se han cogido del año 2012 al 2024 (156 datos en total ya que viene de forma mensual). Por otro lado, solo se ha código la temperatura media mensual:

Año
Total 37
Todos ☐ Invertir Orden ☐ Seleccionados: 13

Mes
Total 12
Todos ☒ Invertir Orden ☐ Seleccionados: 12

2021
2022
2023
2024

Enero
Febrero
Marzo
Abril

Temperatura media y precipitación
Total 2
Todos ☐ Invertir Orden ☐ Seleccionados: 1

Variables de climatología
Total 10
Todos ☐ Invertir Orden ☐ Seleccionados: 1

Temperatura media °C
Precipitación mm

Media mínima
Mínima valor normal
Media mensual
Mensual valor normal

↑ Fila
↓ Fila

Filas

AÑO
MES

»
«

Columnas

CLIMATOLOGIA*
VARIABLE*

↑ Columna
↓ Columna

Las variables con dependencias aparecen con un asterisco(*). Estas deben colocarse en última posición, ya sea en filas o en columnas. Además se moverán juntas al cambiarlas entre filas y columnas.

Una vez tenemos los datos, los descargamos en un excel. Se han codigo estos datos (por ejemplo) ya que se pedía que fuesen datos con estacionalidad (en este caso, tenemos 4 estaciones de las cuales las más diferenciadas son invierno y verano).

Una vez tenemos el excel, lo preparamos eliminando las líneas de relleno que nos ponían al hacer la descarga y se quedaría con el siguiente formato:

	A	B	C
	2012	Enero	5,5
	2012	Febrero	5,1
	2012	Marzo	10,3
	2012	Abril	11,6

Teniendo los datos así en el excel, podríamos comenzar con el tratamiento de estos datos para dejarlos listos para comenzar ccon los siguietnes apartados. Para ello, en python, vamos a convertir las dos columnas en una única con formato datetime y ponerla como índice, mientras dejamos la columna de la media convirtiéndola en float:

```
directorio = 'C:/Users/gerar/Desktop/Master/Master/Mineria de datos - PART II/Tarea/Data'
os.chdir(directorio)
data = pd.read_excel('Media Temperaturas 2012-2024 Madrid.xlsx',header=None)
# print(data)
# ahora con estos datos vamos a darle formato fecha a estas dos primeras ccolumnas convirtiendolo en
una única columna
meses = {
    'Enero': '01',
    'Febrero': '02',
    'Marzo': '03',
    'Abril': '04',
    'Mayo': '05',
    'Junio': '06',
    'Julio': '07',
    'Agosto': '08',
    'Septiembre': '09',
    'Octubre': '10',
}
```

Asignatura: Minería de datos y modelización predictiva – 2

4

```

    'Noviembre':'11',
    'Diciembre':'12'
}
data['Fecha'] = (data[0].astype(str) + '-' + data[1].map(meses))
# print(data)
data['Fecha'] = pd.to_datetime(data['Fecha'], format='%Y-%m')
data['Temp. Media'] = data[2].str.replace(',', '.').astype(float)
data = data.drop([0,1,2], axis=1)
data = data.set_index('Fecha')
print(data)

```

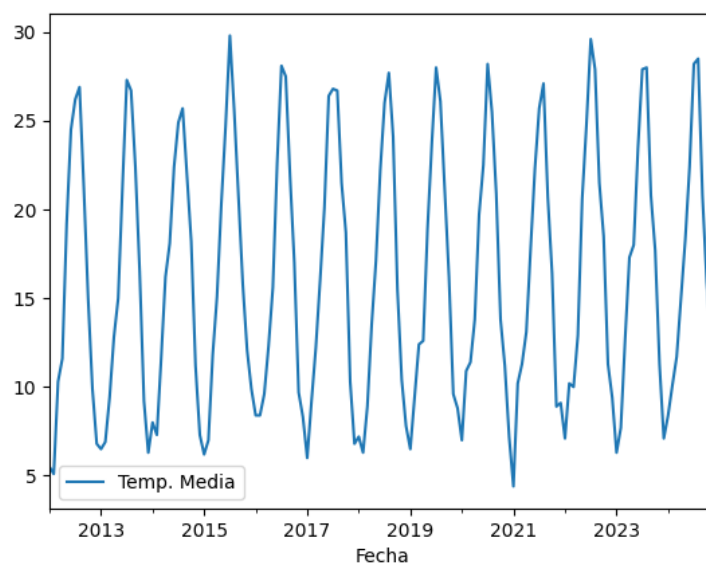
Una vez que se ha ejecutado este código, los datos se quedan de la siguiente forma:

Fecha	Temp. Media
2012-01-01	5,5
2012-02-01	5,1
2012-03-01	10,3
2012-04-01	11,6
2012-05-01	19,2
...	...
2024-08-01	28,5
2024-09-01	20,8
2024-10-01	15,9
2024-11-01	12,5
2024-12-01	7,7

El código usado en este apartado es el mismo que el que se ha visto arriba.

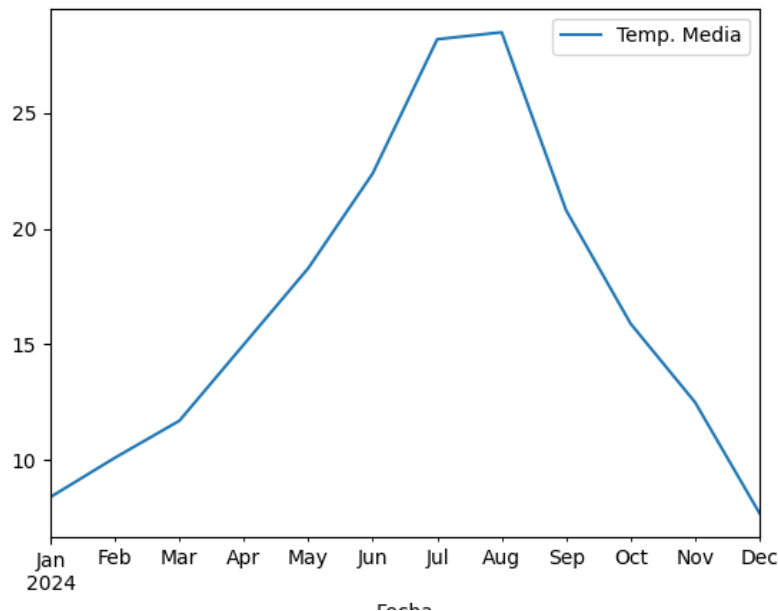
3. Gráficos y comportamiento estacional

Para comenzar, vamos revisar gráficamente los datos que tenemos de forma gráfica:



Como se puede observar gráficamente en esta imagen, se puede ver claramente que esta serie tiene un comportamiento estacional ya que las temperaturas medias suben y bajan (conforme si es verano o es invierno) y esto ocurre todos los años.

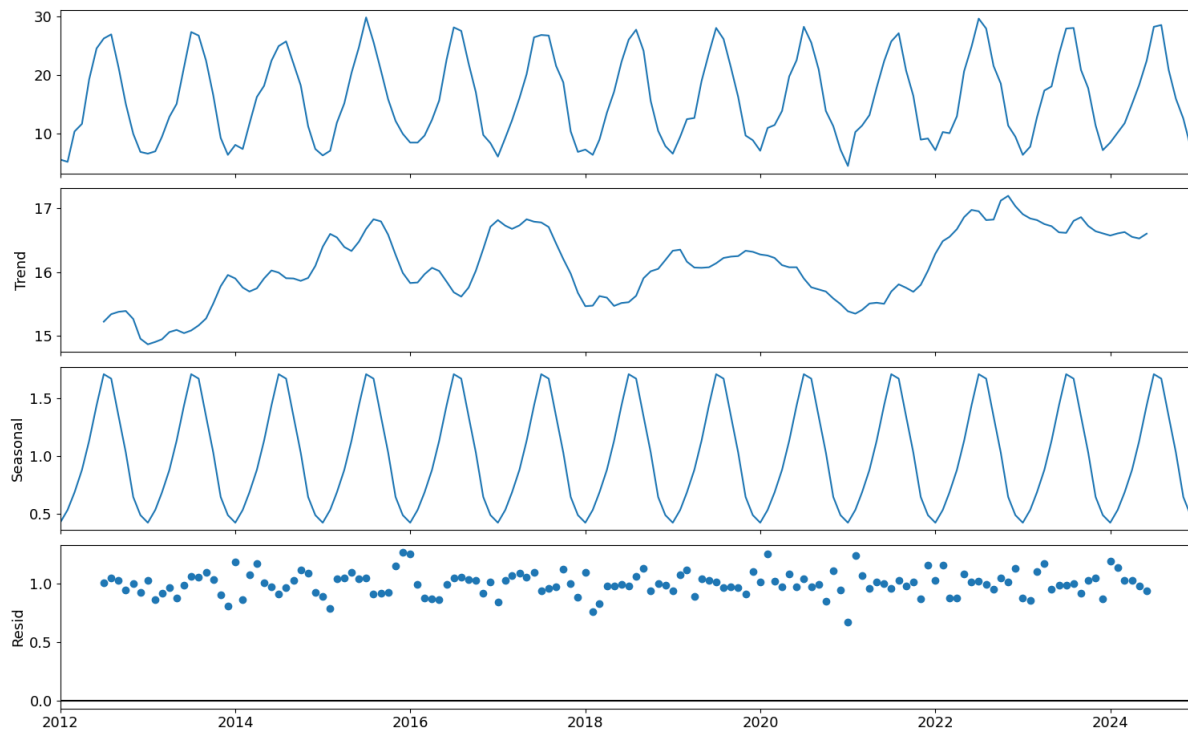
Vamos a centrarnos en el último año de la serie para verla más detenidamente:



Como se puede ver en este último año que tenemos (2024) se puede ver como comienza abajo y luego como va subiendo hasta que llega agosto donde empieza a bajar. Este comportamiento (o uno parecido) es el que tienen todos los años.

Sobre esta serie completa, vamos a realizar la descomposición temporal (con un periodo de 12 que son los meses del año) de forma multiplicativa ya que, como se puede observar en la gráfica, no contiene ni 0 ni valores negativos y lo representamos gráficamente:

```
multiplicative_descompose = seasonal_decompose(data, model='multiplicative', period=12) # ponemos periodo de 12 ya que son los meses
# print(multiplicative_descompose)
plt.rc('figure', figsize=(16,12))
plt.rc('font', size = 13)
fig = multiplicative_descompose.plot()
plt.show()
```



Como se puede observar en esta gráfica, tenemos distintas gráficas que vienen a ser lo siguiente:

- Gráfica superior: Nuestra serie que estamos analizando como se representó anteriormente que es la que hemos descompuesto en las otras gráficas: tendencia, estacionalidad y ruido
- Trend (segunda gráfica): En esta gráfica se puede observar la tendencia de la temperatura en madrid de estos años. Como se puede ver parece tener una tendencia creciente aunque con muchos bajones.
- Estacionalidad (tercera gráfica): Para calcularla se ha eliminado esa tendencia de la serie. Como se puede ver en esta gráfica, los meses de diciembre y enero son sobre todo los valores que están más abajo mientras que los meses de julio y agosto son los que están más arriba ya que es cuando más temperatura hace. Al ser modelo multiplicativo se hace sobre el coeciente con lo que se ve si son mayores que 1 o menores que 1.
- Ruido (cuarta gráfica): Esta gráfica se obtiene eliminando de la serie tanto la tendencia como la estacionalidad. En esta gráfica se ve lo que no se puede controlar ni predecir.

Vamos a ver estos datos del último año :

2024-01-01	0.425152
2024-02-01	0.535895
2024-03-01	0.687647
2024-04-01	0.882279
2024-05-01	1.133850
2024-06-01	1.440143
2024-07-01	1.709437
2024-08-01	1.671126
2024-09-01	1.344170
2024-10-01	1.032037
2024-11-01	0.647675
2024-12-01	0.490590

Como se puede ver en esta tabla, tenemos los meses junto con sus coeficientes estacionales. Se puede ver que cuando son mayor que 1, es cuando más temperatura media hay mientras que cuando es más bajo, menor temperatura media hay.

Por lo tanto, se ve que cuando más temperatura media hay son en los meses de agosto y julio mientras que cuando menos temperatura media hay son en los meses de diciembre y enero.

Código utilizado en este apartado:

```
data.plot()
plt.show()
# vamos a representar un único año apra ver el comportamiento
data_anno = data[-12:]
data_anno.plot()
plt.show()
multiplicative_descompose = seasonal_decompose(data, model='multiplicative',period=12) # ponemos periodo
de 12 ya que son los meses
# print(multiplicative_descompose)
plt.rc('figure', figsize=(16,12))
plt.rc('font',size = 13)
fig = multiplicative_descompose.plot()
plt.show()
print(multiplicative_descompose.seasonal[-12:])
```

4. Dividir datos en train y test

En este apartado, se va a dividir la serie completa en datos de entrenamiento y en datos de testeo. Para los datos de testeo nos vamos a quedar únicamente con la última serie (al ser estacional). Esto quiere decir que nos vamos a quedar con los últimos 12 meses que tenemos (año 2024) mientras que los datos de entrenamiento van a tener el resto de los años (de 2012 a 2023).

Para ello se tiene que usar el siguiente código:

```
train = data[:-12]
test = data[-12:]
```


Aquí hemos dividido los datos en entrenamiento y test:

- Entrenamiento (2012-2023):

2012-01-01	5.5
2012-02-01	5.1
2012-03-01	10.3
2012-04-01	11.6
2012-05-01	19.2
...	...
2023-08-01	28.0
2023-09-01	20.8
2023-10-01	17.7
2023-11-01	11.3
2023-12-01	7.1

- Test (2024):

2024-01-01	8.4
2024-02-01	10.1
2024-03-01	11.7
2024-04-01	15.0
2024-05-01	18.3
2024-06-01	22.4
2024-07-01	28.2
2024-08-01	28.5
2024-09-01	20.8
2024-10-01	15.9
2024-11-01	12.5
2024-12-01	7.7

El código usado en este apartado es el que se muestra anteriormente.

5. Modelos de suavizado

A pesar de tener una tendencia un poco creciente, también vamos a realizar el modelo de suavizado simple. Los modelos de suavizado que vamos a utilizar son los siguientes:

- Modelo de suavizado simple: Los parámetros de este modelo son los siguientes:

===== SimpleExpSmoothing Model Results =====			
Dep. Variable:	Temp. Media	No. Observations:	144
Model:	SimpleExpSmoothing	SSE	2499.820
Optimized:	True	AIC	414.999
Trend:	None	BIC	420.939
Seasonal:	None	AICC	415.287
Seasonal Periods:	None	Date:	Wed, 01 Jul 2026
Box-Cox:	False	Time:	19:04:02
Box-Cox Coeff.:	None		
=====			
	coeff	code	optimized

smoothing_level	1.0000000	alpha	True
initial_level	5.5000048	1.0	True

- Modelo de suavizado doble de holt: Los parámetros de este modelo son los siguientes:

Holt Model Results			
=====			
Dep. Variable:	Temp. Media	No. Observations:	144
Model:	Holt	SSE	1813.681
Optimized:	True	AIC	372.795
Trend:	Additive	BIC	384.675
Seasonal:	None	AICC	373.408
Seasonal Periods:	None	Date:	Wed, 01 Jul 2026
Box-Cox:	False	Time:	19:04:02
Box-Cox Coeff.:	None		
=====			
	coeff	code	optimized

smoothing_level	0.9520155	alpha	True
smoothing_trend	0.9520155	beta	True
initial_level	5.0669722	l.0	True
initial_trend	0.1983475	b.0	True

- Modelo de suavizado de holt-winters: Los parámetros de este modelo son los siguientes:

ExponentialSmoothing Model Results			
=====			
Dep. Variable:	Temp. Media	No. Observations:	144
Model:	ExponentialSmoothing	SSE	245.088
Optimized:	True	AIC	108.580
Trend:	Multiplicative	BIC	156.097
Seasonal:	Multiplicative	AICC	114.052
Seasonal Periods:	12	Date:	Wed, 01 Jul 2026
Box-Cox:	False	Time:	19:04:03
Box-Cox Coeff.:	None		
=====			
	coeff	code	optimized

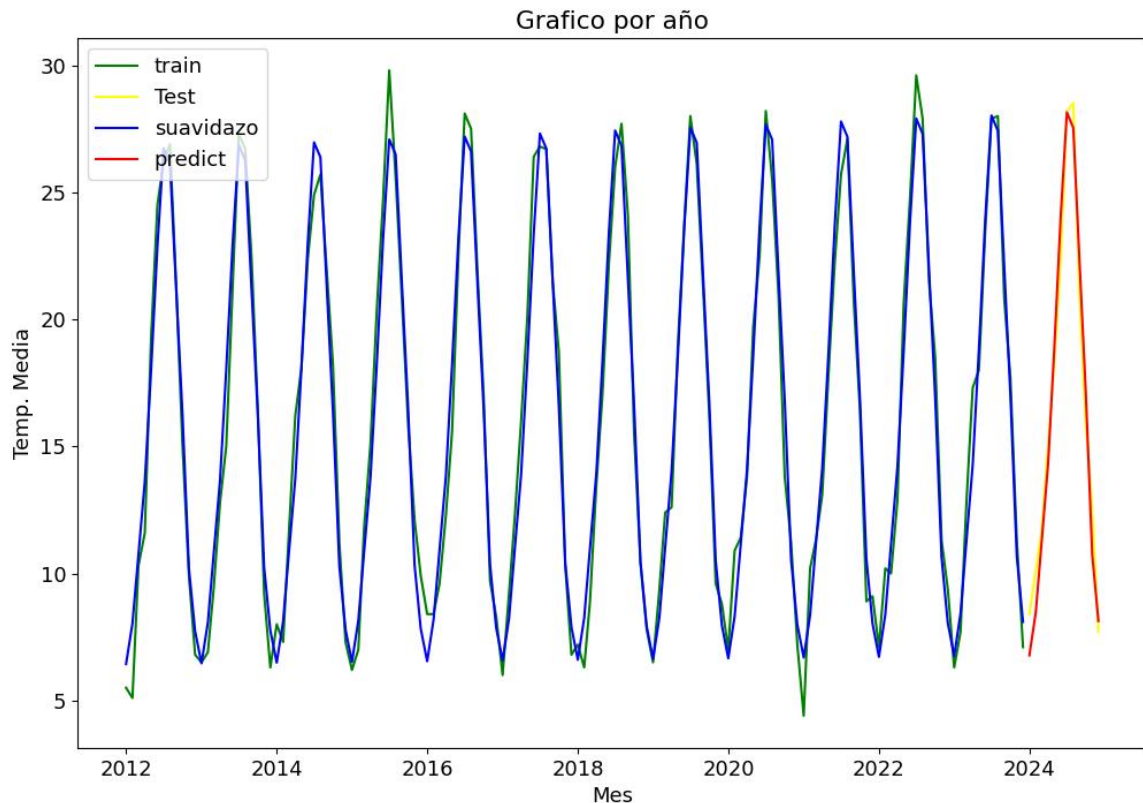
smoothing_level	1.4901e-08	alpha	True
smoothing_trend	1.4302e-09	beta	True
smoothing_seasonal	0.000000	gamma	True
initial_level	15.426739	l.0	True
initial_trend	1.0003573	b.0	True
initial_seasons.0	0.4169071	s.0	True
initial_seasons.1	0.5207813	s.1	True
initial_seasons.2	0.6937225	s.2	True
initial_seasons.3	0.8809833	s.3	True
initial_seasons.4	1.1608089	s.4	True
initial_seasons.5	1.4784608	s.5	True
initial_seasons.6	1.7286753	s.6	True
initial_seasons.7	1.6906429	s.7	True
initial_seasons.8	1.3603908	s.8	True
initial_seasons.9	1.0476376	s.9	True
initial_seasons.10	0.6573760	s.10	True
initial_seasons.11	0.4982312	s.11	True

Para ver que modelo es mejor nos vamos a centrar en los valores de AIC y BIC. Como se puede observar, para el modelo suavizado de holt-winters, es el que tiene estos parámetros con el menor valor de todos los modelos con mucha diferencia. Por este motivo, este es el mejor modelo de los 3. Con lo que creamos los gráficos con las predicciones con este modelo. Para ello usamos el siguiente código:

```
plt.figure(figsize=(12,8))
plt.plot(train,label='train',color='green')
plt.plot(test,label='Test',color='yellow')
plt.plot(model_holt_winters.fittedvalues,label='suavizado',color='blue')
plt.plot(test,label='predict',color='red')
plt.xlabel('Mes')
```

```
plt.ylabel('Temp. Media')
plt.title('Grafico por año')
plt.legend()
plt.show()
```

El gráfico creado es el siguiente:



Como se puede observar en este gráfico tenemos 4 líneas bien diferenciadas:

- Train (Verde): En esta línea se puede observar los datos que hemos utilizado de entrenamiento que son los datos reales
- Test (Amarillo): En esta línea se puede observar los datos que hemos utilizado para el testeo que son también datos reales
- Suavizado (Azul): Estos son los datos reales de entrenamiento a los que se le ha aplicado el método de suavizado (se puede observar que son casi los mismos que los datos reales menos por unos picos más altos y bajos)
- Predict (Rojo): Estos son los datos que se han predicho para el último año. Como se puede observar, estos datos coinciden bastante bien con los datos reales con lo que se podría decir que el modelo ha realizado una buena predicción.

Como se puede observar con este modelo, es un modelo bastante adecuado ya que los datos de test coinciden de forma bastante cercana con los valores que se han predicho.

Las predicciones que se han realizado son las siguientes:

2024-01-01	6.773423
2024-02-01	8.464072
2024-03-01	11.278853
2024-04-01	14.328540
2024-05-01	18.886443
2024-06-01	24.063255
2024-07-01	28.145769
2024-08-01	27.536372
2024-09-01	22.165302
2024-10-01	17.075607
2024-11-01	10.718502
2024-12-01	8.126551

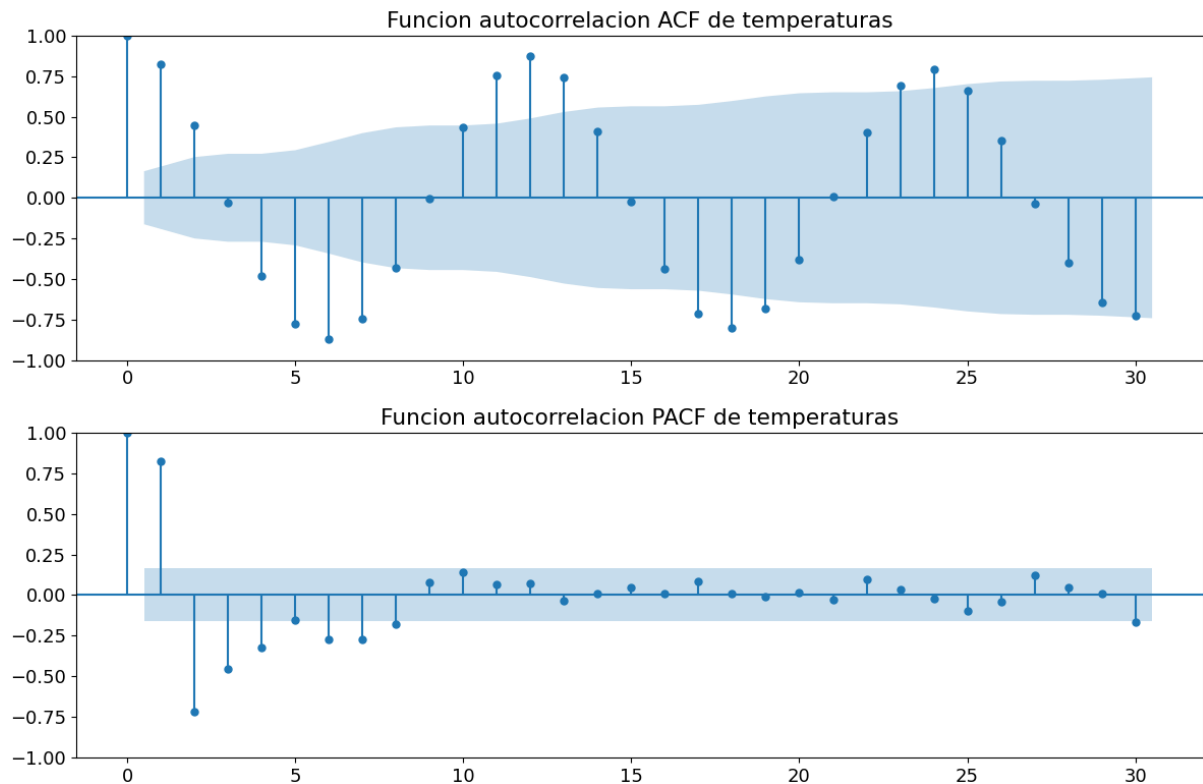
El código utilizado en este apartado es el siguiente:

```
model_simple = SimpleExpSmoothing(train,initialization_method='estimated').fit() ## simple
print(model_simple.summary())
model_holt = Holt(train, initialization_method='estimated').fit()
print(model_holt.summary())
model_holt_winters = ExponentialSmoothing(train,seasonal_periods=12,trend='mul',
                                         seasonal='mul', initialization_method='estimated'
                                         ).fit() ## multiplicativo -> holt_winters
print(model_holt_winters.summary())
f_cast = model_holt_winters.forecast(12)
plt.figure(figsize=(12,8))
plt.plot(train,label='train',color='green')
plt.plot(test,label='Test',color='yellow')
plt.plot(model_holt_winters.fittedvalues,label='suavizado',color='blue')
plt.plot(f_cast,label='predict',color='red')
plt.xlabel('Mes')
plt.ylabel('Temp. Media')
plt.title('Grafico por año')
plt.legend()
plt.show()
print(f_cast)
```

6. Correlogramas y modelo Arima manual

En este apartado vamos a comenzar calculando los correlogramas de la serie normal con los datos de train.

Este código genera la siguiente gráfica:



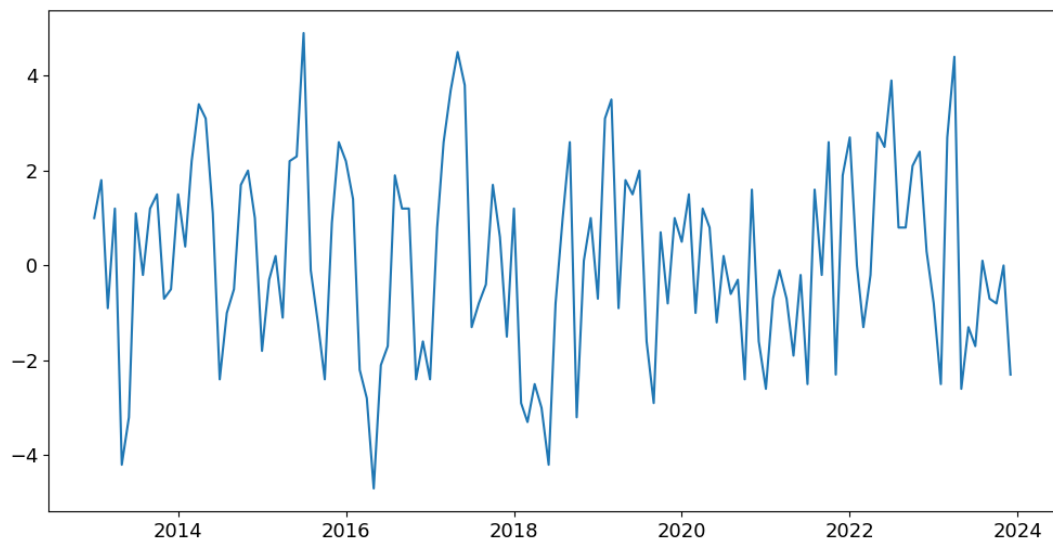
En estas gráficas se deben ignorar siempre la posición 0 ya que siempre valdrán 1, con lo que no se tienen que tener en cuenta.

En estas gráficas se puede observar que en la autocorrelación ACF tiene un comportamiento periódico ya que sube y baja todo el rato de una forma muy similar, lo que nos quiere decir que es una serie estacionaria.

Ahora vamos a realizar la diferenciación de la serie ya que tiene mucha diferencia entre los meses de diciembre y enero con los de julio y agosto. Para calcular las diferencias se usa el siguiente código (mostramos gráficamente esta serie diferenciada):

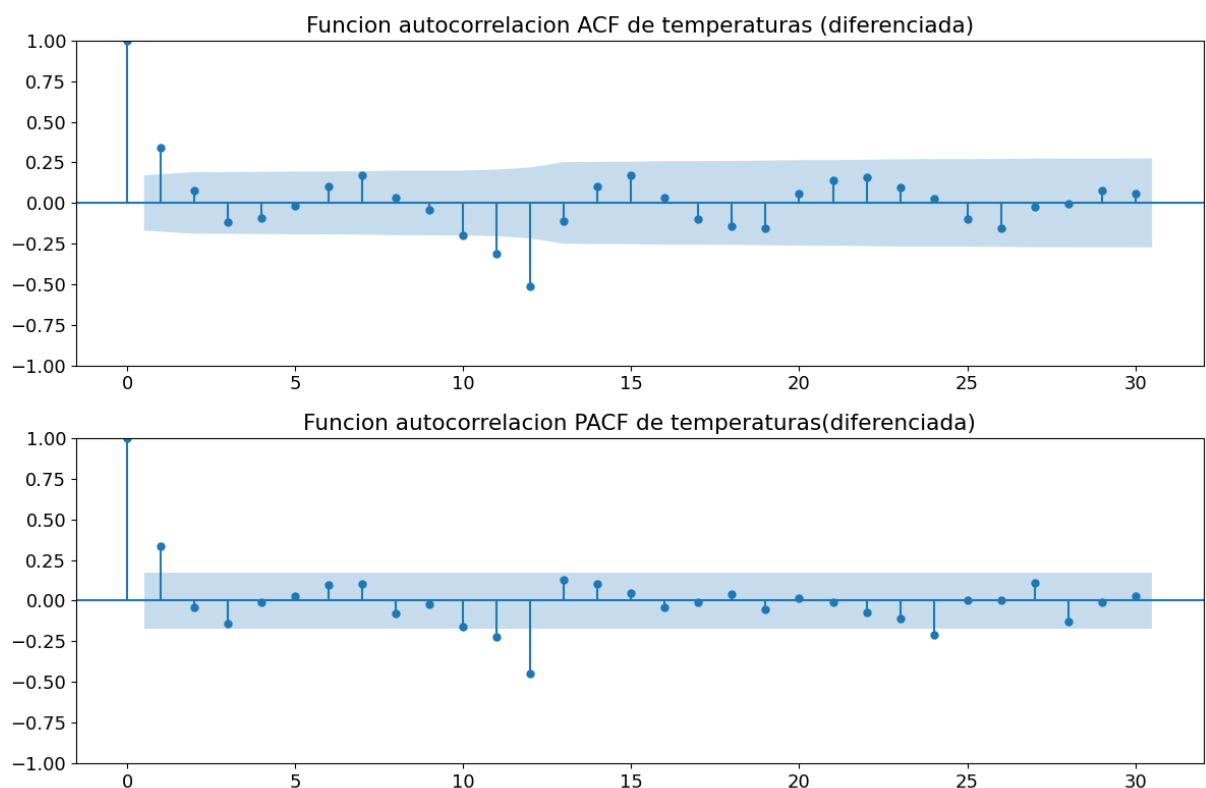
```
diferencias = train.diff(12)# calculamos las diferencias
# grafica
plt.figure(figsize=(12,6))
plt.plot(diferencias)
plt.show()
```

La gráfica de la serie diferenciada es la siguiente:



Una vez hecho a la diferenciación podemos observar que ahora la gráfica y la media de la gráfica tiende a ser 0, ya que hemos eliminado los picos altos y bajos haciendo la diferenciación.

Una vez tenemos así la serie diferenciada, creamos las gráficas de los correlogramas ACF y PACF:



En estas gráficas de ahora podemos observar que los valores están más igualados y más dentro de la banda. En la gráfica de arriba podemos observar que sigue teniendo un comportamiento periódico mientras que en la gráfica de abajo vemos que en el

punto 1, está muy por encima mientras que en el punto 12, está muy abajo, cosa que pasa igual en el punto 24.

Por lo tanto, para crear un modelo ARIMA de forma manual, debemos tener en cuenta lo siguiente:

- Vemos que el modelo ACF decrece de forma rápida
- Vemos que la autocorrelación de orden uno de la gráfica PACF, vemos que tiene un valor muy fuerte y luego en la 12 también se repite esto.

Por lo tanto, para crear nuestro modelo arima manual, vamos a poner en la parte regular un $p = 1$ mientras que en la parte estacional vamos a ponerle un $Q = 1$ y vamos a realizar la diferenciación en esta parte ($D = 1$).

Con lo que se quedaría con el siguiente modelo:

ARIMA(1,0,0)(0,1,1)12

Para ver los datos de los coeficientes y ver gráficamente este modelo manual que hemos creado, tenemos que crear el modelo y posteriormente ver la diagnosis del modelo para ver que tenemos un modelo decente.

Para ello se tiene que usar el siguiente código:

```
modelo_arima = sm.tsa.ARIMA(train, order= (1,0,0), seasonal_order=(0,1,1,12))
resultados = modelo_arima.fit()
print(resultados.summary())
resultados.plot_diagnostics(figsize=(12,8))
plt.show()
```

Primero, vamos a estudiar la tabla que se ha generado que es la siguiente:

SARIMAX Results

Dep. Variable:Temp. MediaNo. Observations:144

Model:ARIMA(1, 0, 0)x(0, 1, [1], 12)Log Likelihood:-239.928

Date:Wed, 01 Jul 2026AIC485.856

Time:22:05:43BIC494.505

Sample:01-01-2012HQIC489.570

- 12-01-2023

Covariance Type:opg

	coef	std err	z	P> z	[0.025	0.975]
ar.L1	0.2872	0.082	3.516	0.000	0.127	0.447
ma.S.L12	-0.8863	0.111	-7.983	0.000	-1.104	-0.669
sigma2	1.9386	0.294	6.592	0.000	1.362	2.515

Ljung-Box (L1) (Q):0.28Jarque-Bera (JB):2.47

Prob(Q):0.60Prob(JB):0.29

Heteroskedasticity (H):1.04Skew:0.05

Prob(H) (two-sided):0.89Kurtosis:2.34

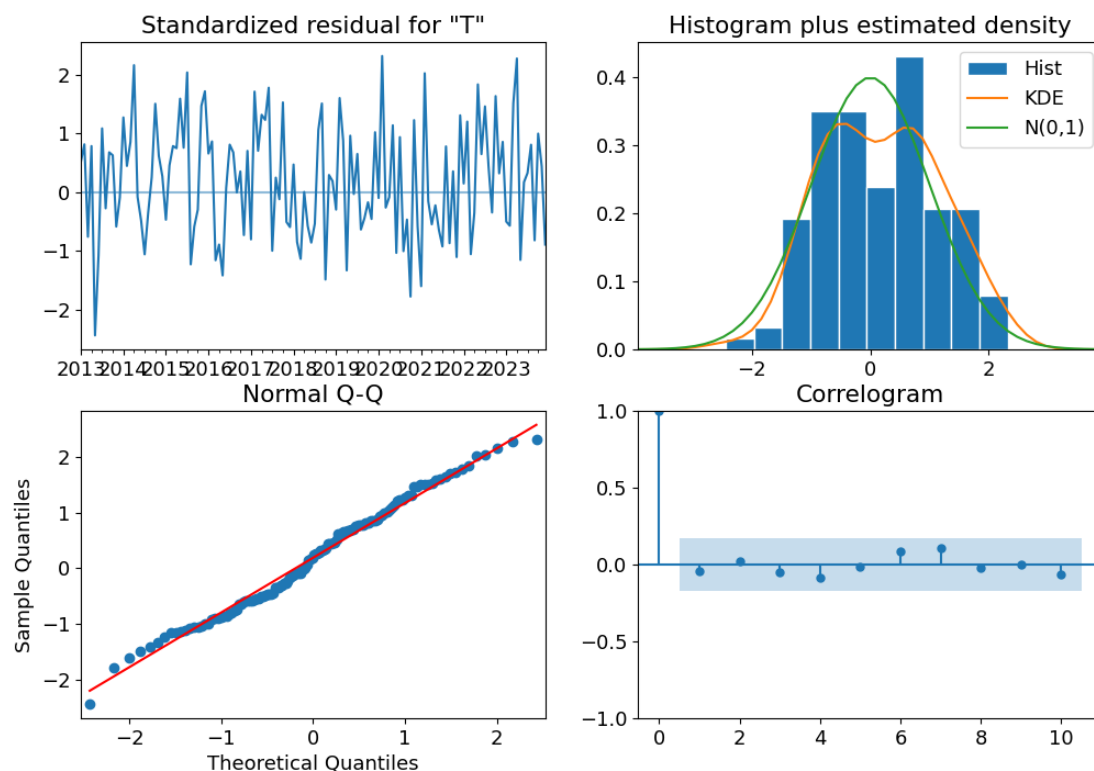
En esta tabla que hemos generado podemos ver que los coeficientes que tenemos son significativos (parte amarilla) con lo que podemos decir que la parte de autorregresión tiene un coeficiente de 0.2872 mientras que la parte de medias móviles tiene un coeficiente de -0.8863.

Además, podemos decir al 95% de confianza que estos coeficientes son significativos debido a que el Pvalor es 0 (parte roja).

En esta tabla también podemos observar los valores de Jung Box (parte verde) con un valor de 0.28 con un valor de confianza de 0.6. Con lo que estamos aceptando que los residuos están incorrelados.

Por último, debemos tener en cuenta también los valores AIC y BIC (parte rosa) que son bastante bajos también, pero tendríamos que ver otros modelos ARIMA para ver si existieran opciones mejores. Ya que cuanto más bajos estos valores, mejor será el modelo.

Ahora, gráficamente podemos ver lo siguiente:



En estas gráficas podemos observar lo siguiente:

- En la gráfica superior izquierda, podemos ver que la media de la serie está muy cerca de 0 con lo que esto es bueno para el modelo ya que es uno de los requisitos.
- Además, podemos ver en la gráfica superior derecha que tiene un comportamiento normal ya que la parte verde se asemeja mucho a la parte naranja.

- Por otro lado, también podemos ver que tiene un comportamiento normal en la gráfica inferior izquierda ya que sigue una recta lineal con solo un par de valores que se han ido un poco. Esto es otro de los requisitos.
- Por último, en la gráfica del correlograma, podemos observar que todos los valores se encuentran dentro de la banda de los coeficientes límites, el cual es otro de los requisitos.

Viendo esto (tanto lo de la tabla como lo de las gráficas) podríamos decir que los residuos están incorrelados.

Para este apartado se ha usado el siguiente código:

```
figure,(ax1,ax2) = plt.subplots(2,1,figsize=(12,8))
plot_acf(train, lags=30, ax=ax1) ## aqui calculamos las simples
ax1.set_title('Funcion autocorrelacion ACF de temperaturas')
plot_pacf(train,lags=30,ax=ax2) ## aqui calculamos las parciales
ax2.set_title('Funcion autocorrelacion PACF de temperaturas')
plt.tight_layout()
plt.show()
diferencias = train.diff(12)# calculamos las diferencias
# grafica
plt.figure(figsize=(12,6))
plt.plot(diferencias)
plt.show()
diferencias = diferencias.dropna()
figure1,(ax11,ax22) = plt.subplots(2,1,figsize=(12,8))
plot_acf(diferencias, lags=30, ax=ax11) ## aqui calculamos las simples
ax11.set_title('Funcion autocorrelacion ACF de temperaturas (diferenciada)')
plot_pacf(diferencias,lags=30,ax=ax22) ## aqui calculamos las parciales
ax22.set_title('Funcion autocorrelacion PACF de temperaturas(diferenciada)')
plt.tight_layout()
plt.show()
modelo_arima = sm.tsa.ARIMA(train, order= (1,0,0), seasonal_order=(0,1,1,12))
resultados = modelo_arima.fit()
print(resultados.summary())
resultados.plot_diagnostics(figsize=(12,8))
plt.show()
```

7. Modelo ARIMA automático

Ahora, para ver cuál sería el mejor modelo arima, vamos a ver los distintos modelos posibles que se pueden formar para quedarnos con el mejor.

Para ello, se ha tenido que poner el siguiente código:

```
modelo_auto =
pm.auto_arima(train,start_p=1,start_q=1,max_p=3,max_1=3,m=12,start_P=0,seasonal=True,d=0,D=1,
              trace=True,error_action='ignore', suppress_warnings=True, stepwise=True)
```

En este código podemos ver que hemos puesto una q y p de inicio de 1 y un máximo de 3 y sin diferencia ya que no se debe hacer este modelo de forma muy complicada (esto

en la parte regular). Por otro lado, en la parte estacional se ha puesto un comienzo de P de 0 y una diferenciación de 1. Con esto, nos salen todos los siguientes modelos:

```
ARIMA(1,0,1)(0,1,1)[12] intercept : AIC=inf, Time=0.34 sec
ARIMA(0,0,0)(0,1,0)[12] intercept : AIC=565.489, Time=0.01 sec
ARIMA(1,0,0)(1,1,0)[12] intercept : AIC=511.715, Time=0.05 sec
ARIMA(0,0,1)(0,1,1)[12] intercept : AIC=inf, Time=0.17 sec
ARIMA(0,0,0)(0,1,0)[12] : AIC=563.965, Time=0.01 sec
ARIMA(1,0,0)(0,1,0)[12] intercept : AIC=551.379, Time=0.02 sec
ARIMA(1,0,0)(2,1,0)[12] intercept : AIC=496.255, Time=0.12 sec
ARIMA(1,0,0)(2,1,1)[12] intercept : AIC=486.522, Time=0.28 sec
ARIMA(1,0,0)(1,1,1)[12] intercept : AIC=484.568, Time=0.11 sec
ARIMA(1,0,0)(0,1,1)[12] intercept : AIC=inf, Time=0.22 sec
ARIMA(1,0,0)(1,1,2)[12] intercept : AIC=486.544, Time=0.57 sec
ARIMA(1,0,0)(0,1,2)[12] intercept : AIC=484.545, Time=0.18 sec
ARIMA(0,0,0)(0,1,2)[12] intercept : AIC=inf, Time=0.18 sec
ARIMA(2,0,0)(0,1,2)[12] intercept : AIC=486.368, Time=0.20 sec
ARIMA(1,0,1)(0,1,2)[12] intercept : AIC=486.420, Time=0.30 sec
ARIMA(0,0,1)(0,1,2)[12] intercept : AIC=485.366, Time=0.15 sec
ARIMA(2,0,1)(0,1,2)[12] intercept : AIC=488.335, Time=0.32 sec
ARIMA(1,0,0)(0,1,2)[12] : AIC=486.535, Time=0.14 sec
```

Best model: ARIMA(1,0,0)(0,1,2)[12] intercept

Viendo estos modelos, podemos observar que el mejor modelo es el siguiente:

ARIMA(1,0,0)(0,1,2) ya que es el que menos valor de AIC tiene.

Por otro lado, si lo comparamos con nuestro modelo (el cual el modelo automático no ha podido generarlo) sigue siendo mejor este que el manual ya que el AIC del modelo manual era de 485.856 mientras que el de este modelo es de 484.545. Por lo tanto, nos quedamos con el modelo que nos ha dado el automático.

Para este apartado se ha usado el código que se muestra anteriormente.

8. Expresión algebraica

En este apartado se nos pide calcular la expresión algebraica de este modelo ARIMA hallando X_{t-k} . Para ello, hemos necesitado generar la siguiente tabla:

SARIMAX Results

Dep. Variable:Temp. MediaNo. Observations:144

Model:ARIMA(1, 0, 0)x(0, 1, [1, 2], 12)Log Likelihood-237.272

Date:Wed, 01 Jul 2026AIC484.545

Time:22:49:21BIC498.959

Sample:01-01-2012HQIC490.402

- 12-01-2023

Covariance Type:opg

	coef	std err	z	P> z	[0.025	0.975]
x1	0.0075	0.004	2.127	0.033	0.001	0.014
ar.L1	0.2328	0.086	2.710	0.007	0.064	0.401
ma.S.L12	-0.9838	0.099	-9.909	0.000	-1.178	-0.789
ma.S.L24	0.1242	0.119	1.042	0.297	-0.109	0.358
sigma2	1.8651	0.297	6.275	0.000	1.283	2.448

Ljung-Box (L1) (Q):0.01Jarque-Bera (JB):2.88

Prob(Q):0.94Prob(JB):0.24

Heteroskedasticity (H):1.08Skew:0.11

Prob(H) (two-sided):0.80Kurtosis:2.31

Una vez que tenemos esta tabla con estos valores, se ha resultado de la siguiente forma:

$$\text{Arima}(1,0,0)(0,1,2) \Rightarrow (1 - 0.2328B)(1 - B^{12})X_t = (1 + 0.9838B^{12} + 0.1242B^{24})Z_t$$

Teniendo en cuenta los valores de la tabla, quedaría de la siguiente forma:

$$\Rightarrow (1 - 0.2328B)(1 - B^{12})X_t = (1 + (-0.9838B^{12}) + 0.1242B^{24})Z_t \Rightarrow$$

$$\Rightarrow (1 - 0.2328B) \cancel{(1 - B^{12})} (X_t - X_{t-12}) = (Z_t - 0.9838Z_{t-12} + 0.1242Z_{t-24}) \Rightarrow$$

$$\Rightarrow X_t \Rightarrow 0.2328X_{t-1} + X_{t-12} + 0.2328X_{t-13} + Z_t + 0.9838Z_{t-12} + 0.1242Z_{t-24}$$

El código utilizado en este apartado es el mismo que se ha usado en el siguiente.

9. Predicciones del modelo ARIMA

Una vez que tenemos el mejor modelo ARIMA, creamos la tabla para ver sus valores:

SARIMAX Results

Dep. Variable:Temp. MediaNo. Observations:144

Model:ARIMA(1, 0, 0)x(0, 1, [1, 2], 12)Log Likelihood-237.272

Date:Wed, 01 Jul 2026AIC484.545

Time:22:49:21BIC498.959

Sample:01-01-2012HQIC490.402

- 12-01-2023

Covariance Type:opg

	coef	std err	z	P> z	[0.025	0.975]
x1	0.0075	0.004	2.127	0.033	0.001	0.014
ar.L1	0.2328	0.086	2.710	0.007	0.064	0.401
ma.S.L12	-0.9838	0.099	-9.909	0.000	-1.178	-0.789
ma.S.L24	0.1242	0.119	1.042	0.297	-0.109	0.358
sigma2	1.8651	0.297	6.275	0.000	1.283	2.448

Ljung-Box (L1) (Q):0.01Jarque-Bera (JB):2.88

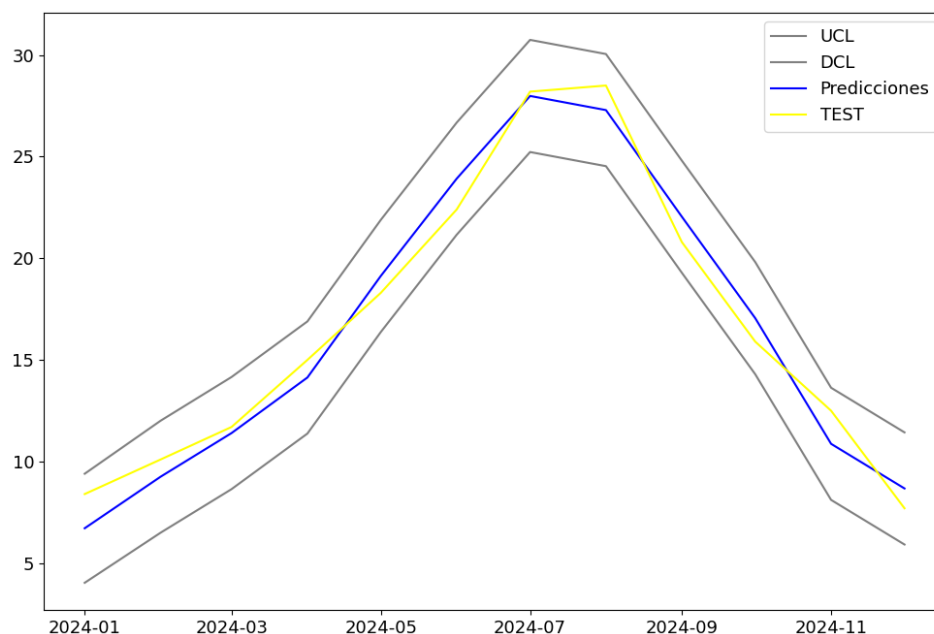
Prob(Q):0.94Prob(JB):0.24

Heteroskedasticity (H):1.08Skew:0.11

Prob(H) (two-sided):0.80Kurtosis:2.31

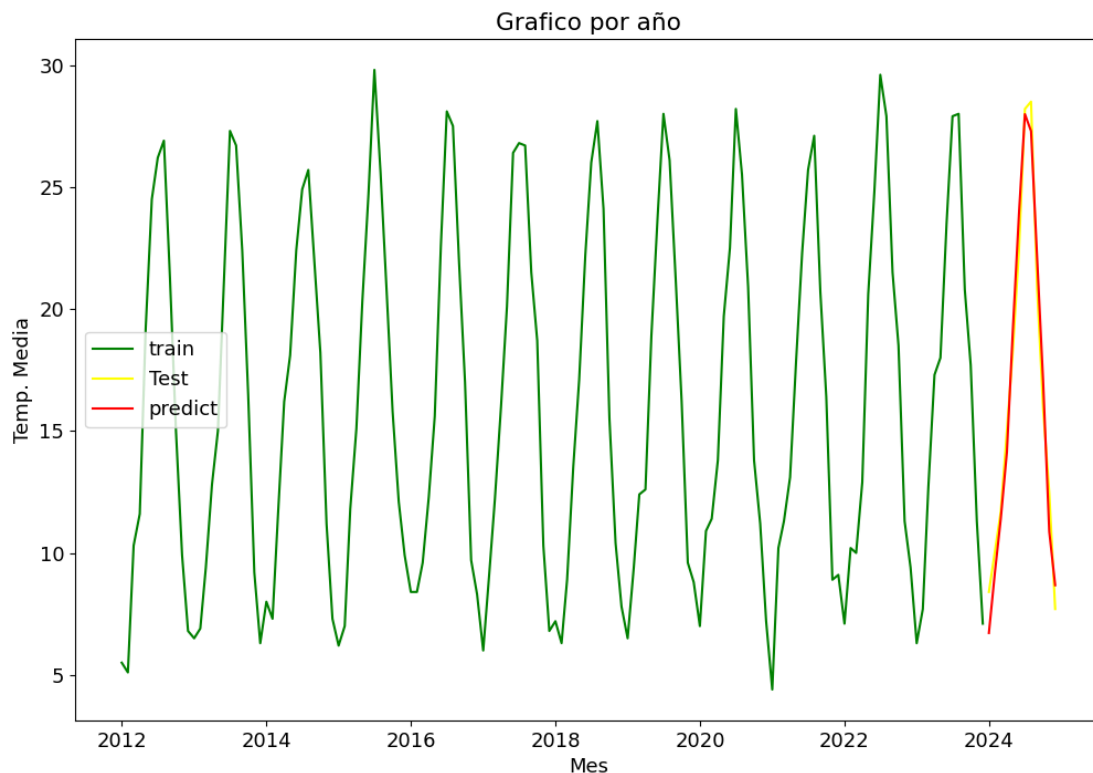
Como se puede observar en esta tabla este modelo tiene el AIC que se indicaba anteriormente que es muy bajo y además tiene además un p valor muy bajo (dentro del 95% de confianza) de los coeficientes.

Representando gráficamente junto con los valores test y los intervalos de confianza vemos lo siguiente:



Con esta gráfica podemos observar que tanto la predicción como los valores de prueba se encuentran dentro de los límites del intervalo de confianza y que, además, se parecen mucho los valores predichos y los valores de los test.

Viéndolo con la serie completa quedaría de la siguiente forma:



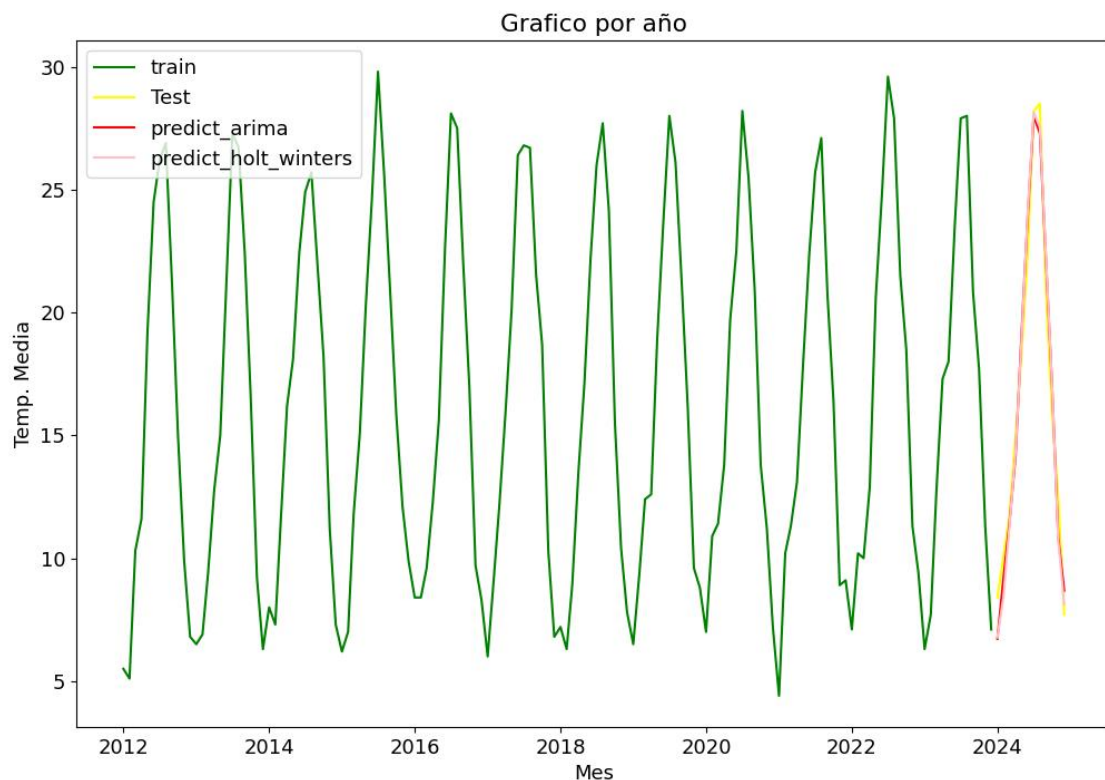
En este apartado se ha utilizado el siguiente código:

```
best_arima = sm.tsa.ARIMA(train,order=(1,0,0),seasonal_order=(0,1,2,12),trend='t')
restultados_best = best_arima.fit()
prediciones_A = restultados_best.get_forecast(steps=12)
predi_test = prediciones_A.predicted_mean
intervalos_confianza_A = prediciones_A.conf_int()
print(restultados_best.summary())
plt.figure(figsize=(12,8))
plt.plot(intervalos_confianza_A['lower Temp. Media'],label='UCL',color='gray')
plt.plot(intervalos_confianza_A['upper Temp. Media'],label='DCL',color='gray')
plt.plot(prediciones_A.predicted_mean, label='Predicciones',color='blue') #datos de prediccion
plt.plot(test,label='TEST', color='yellow') # datos reales
plt.legend()
plt.show()
plt.figure(figsize=(12,8))
plt.plot(train,label='train',color='green')
plt.plot(test,label='Test',color='yellow')
plt.plot(prediciones_A.predicted_mean,label='predict',color='red')
plt.xlabel('Mes')
plt.ylabel('Temp. Media')
plt.title('Grafico por año')
plt.legend()
plt.show()
```

10. Comparación ARIMA y suavizado

Como hemos visto previamente en las gráficas, se puede decir que tanto con el modelo suavizado de holt-winters y el modelo ARIMA, nos hemos acercado mucho con las predicciones a los valores reales.

Con lo que, comparando estas dos modelos, quedaría la gráfica de la siguiente forma:



Como se puede observar en esta gráfica, a simple ojo es muy complicado diferenciar cuál podría ser mejor modelo (si el suavizado o el modelo ARIMA) ya que ambos son muy parecidos y se asemejan mucho a los valores test.

Por lo tanto viendo los valores del AIC y del BIC podríamos decir que el modelo de HOLT-WINTERS es mejor que el modelo ARIMA debido a que los valores de estos parámetros son bastante menores que el del ARIMA.

En conclusión, el mejor modelo para estos datos se trataría del modelo HOLT-WINTERS.

Para este último apartado se ha usado el siguiente código:

```
plt.figure(figsize=(12,8))
plt.plot(train,label='train',color='green')
plt.plot(test,label='Test',color='yellow')
plt.plot(predicciones_A.predicted_mean,label='predict_arima',color='red')
plt.plot(f_cast,label='predict_holt_winters',color='pink')
plt.xlabel('Mes')
```

```
plt.ylabel('Temp. Media')  
plt.title('Grafico por año')  
plt.legend()  
plt.show()
```